Battery MCP Server — Design Plan (v2)

Status: Draft for domain-expert review **Audience:** Battery domain expert + AssetOpsBench maintainer **Goal:** Add a dedicated `battery` MCP server to AssetOpsBench that uses the pretrained acctouhou Li-ion model for RUL and voltage prediction, plus statistical tools for scenarios the pretrained model can't cover.

---

1. Problem statement

AssetOpsBench currently handles chillers (IoT sensor data) and rotating machinery (vibration DSP). An attempt to add battery analytics by extending the generic TSFM server failed because:

- Battery data is cycle-indexed, not continuous-time — TSFM's context-length requirement rejected the data structurally. See §1.1 for exactly what this means.
- We pre-aggregated raw cycle time-series into one-row-per-cycle summaries, throwing away the within-cycle voltage/current/temperature curves that real battery models consume.
- Battery analytics requires domain concepts (SOH, knee point, Coulombic efficiency, DCIR growth) that are foreign to a generic foundation model.

Solution: a dedicated `battery` MCP server, following the vibration server pattern. Uses the **acctouhou pretrained Li-ion model** (Hsu et al., *Applied Energy* 2022, doi: 10.1016/j.apenergy.2021.118134) for RUL and voltage prediction; uses statistical methods for impedance growth and anomaly detection.

1.1 What TSFM's `context_length` actually means

Verified directly from the model configs in `src/servers/tsfm/artifacts/tsfm_models/*/config.json` and the inference assertion in `src/servers/tsfm/forecasting.py:243`:

- `context_length` is the **number of consecutive timesteps = consecutive rows** in the input DataFrame that the model consumes to produce a forecast.
- Not tokens, not characters, not JSON entries. For TTM architecturally: `context_length = num_patches × patch_length`. The default `ttm_96_28` has `num_patches=12, patch_length=8 → 96`.
- When the server is called with `id_columns=["asset_id"]`, each asset becomes its own time-series segment. Each segment must have `≥ context_length` contiguous rows after the data-quality filter drops NaN rows and breaks segments at timestamp discontinuities (`forecasting.py:94–96`: `group_sizes[group_sizes >= n_minimum]`).
- For fine-tuning, the minimum rises to `context_length + prediction_length` (need ground truth for the horizon).
- Segments shorter than the minimum are **silently dropped**, which is why our earlier battery experiments looked like "TSFM returned nothing."

The repo ships with multiple TTM checkpoints, not just the 96-step one:

Checkpoint	`context_length`	`prediction_length`	Min rows (inference)	Min rows (finetune)

Checkpoint	`context_length`	`prediction_length`	Min rows (inference)	Min rows (finetune)
`ttm_52_16_mae`	52	16	52	68
`ttm_90_30_mse` / `ttm_90_30_mae`	90	30	90	120
`ttm_96_28` (planner default)	**96**	**28**	**96**	**124**
`ttm_512_96`	512	96	512	608

For battery scenarios where we want to use TSFM at all (e.g. scenario 13 fallback, scenario 15 benchmark), picking the right checkpoint matters. Our 10 "usable" cells all have 102–197 discharge cycles, so they clear 96 for inference but several fail the 124-row floor for fine-tuning. `ttm_52_16_mae` would let more short-cycle cells participate at the cost of shorter context.

---

2. Dataset scope

Full raw NASA dataset: 34 unique cells across 6 batches.

After filtering by the acctouhou model's constraints (≥100 discharge cycles per cell, each cycle ≥100 samples), **10 cells are directly usable for the pretrained model:**

Cell	Discharge cycles	Ambient °C	V cutoff	Discharge rate	Capacity fade
B0005	168	24	2.7 V	2 A	28.6%
B0006	168	24	2.5 V	2 A	41.7%
B0007	168	24	2.2 V	2 A	24.3%
B0018	132	24	2.5 V	2 A	27.7%
B0033	197	24	2.0 V	4 A	varies
B0034	197	24	2.2 V	4 A	varies
B0036	197	24	2.7 V	2 A	varies
B0054	102	4	2.2 V	2 A	—
B0055	102	4	2.5 V	2 A	—
B0056	102	4	2.7 V	2 A	—

The other 24 cells are either too short (≤72 cycles, mostly B0025–B0032, B0038–B0040, B0045–B0048) or have NASA-acknowledged quality issues (B0041–B0044, B0049–B0053 have 25–63% of discharge runs truncated to <100 samples due to cold-temperature early termination). These 24 cells are **still usable for the non-model statistical tools** (impedance growth, capacity fade, outlier detection) because those don't require 100-cycle windows or 500-sample interpolation.

---

3. Pretrained model — the acctouhou pipeline

Source: https://github.com/acctouhou/Prediction_of_battery Trained on: Severson et al. Stanford/Toyota LFP 18650 dataset (~124 cells) Weights: `feature_selector_ch.h5`, `feature_selector_dis.h5`, `predictor.h5` (RUL), `predictor2.h5` (voltage curves). Hosted on Google Drive per the repo README.

Exact input tensor spec (ground truth from `data_processing.ipynb`)

Per cell, per track (charge or discharge), the model expects:

```
shape:   (n_cycles, 4, 500)                # before normalization
channels: [Q, V, I, T]                     # in this exact order
         Q = cumulative capacity curve (Ah)
         V = voltage (V)
         I = current (A)
         T = temperature (°C)
timesteps: 500 via scipy linear interpolation
cycles:  first 100 per cell
```

The feature-selector step does `np.transpose(x, (0, 2, 1))` internally, so the model's actual input is `(n_cycles, 500, 4)` after normalization:

```python
# From acctouhou predict.py (verbatim):
def feature_selector(model, x, norm):
    normalized_data = (np.transpose(x, (0, 2, 1)) - norm[0]) / norm[1]
    return model.predict(normalized_data, batch_size=128)
```

Full inference pipeline

```
1. Raw NASA JSON cycles
     ↓ preprocessing (see §4)
2. charge_data:    (n_cells, 100, 4, 500)
   discharge_data: (n_cells, 100, 4, 500)
   summary_data:   (n_cells, 100, S)           # S = summary feature dim,
see §4.3
     ↓ normalize with charge_norm.npy / discharge_norm.npy /
summary_norm.npy
     ↓ feature_selector_ch.predict(...)  → (n_cells, 100, F_ch)
     ↓ feature_selector_dis.predict(...) → (n_cells, 100, F_dis)
     ↓ hstack with normalized summary → (n_cells, 100, F_ch + F_dis + S) =
(n_cells, 100, 12)
3. cell_feature[i]: (100, 12) per cell
     ↓ sliding-window (process2predict)
     ↓ edge-padding to 50 cycles per window
4. x_in: (n_windows, 50, 12)
     ↓ predictor.predict(...) → (n_windows, 2)
5. Output: [RUL_normalized, S_normalized]
     ↓ * predict_renorm[:,1] + predict_renorm[:,0]
6. Final: [RUL_cycles, start_cycle_index]
```

Model outputs

- `predictor.h5`: **(RUL, S)** — remaining useful life in cycles + current start cycle index
- `predictor2.h5`: **voltage-vs-SOC curve + capacity curve + power** across the predicted remaining lifetime

What we download from their Google Drive

```
pretrained/
    feature_selector_ch.h5     # charge-side feature extractor
    feature_selector_dis.h5    # discharge-side feature extractor  (uses
custom 'mish' activation)
    predictor.h5               # RUL head (uses 'mish')
    predictor2.h5              # voltage/capacity/power head (uses 'mish')
dataset/
    charge_norm.npy            # mean/scale for charge channels
    discharge_norm.npy         # mean/scale for discharge channels
    summary_norm.npy           # mean/scale for summary features
    predict_renorm.npy         # mean/scale for denormalizing RUL + S
outputs
```

We do NOT need their `charge_data.npy` / `discharge_data.npy` / `summary_data.npy` / `battery_EoL.npy` / `index_battery.npy` — those are their Severson-preprocessed tensors, which we replace with NASA-preprocessed equivalents.

---

4. Preprocessing: raw NASA JSON → acctouhou input tensors

This is the bridge. Writing the explicit pseudocode below so the domain expert can sanity-check the physics.

4.1 Raw NASA JSON structure (confirmed by reading the files)

```
{
  "B0018": {
    "cycle": [
      {
        "type": "charge",
        "ambient_temperature": 24,
        "time": [2008, 5, 30, 10, 23, 42.094],
        "data": {
          "Voltage_measured": [...],   # volts
          "Current_measured": [...],   # amps
          "Temperature_measured": [...], # °C
          "Current_charge": [...],
          "Voltage_charge": [...],
          "Time": [...]                # seconds within the cycle
        }
      },
      { "type": "discharge", ..., "data": { ..., "Capacity": 1.856 } },
      { "type": "impedance", ..., "data": { "Re": 0.075, "Rct": 0.142, ... }
```

```
  },
        ...
      ]
    }
  }
```

Cycle types alternate in order: charge, discharge, (sometimes impedance), charge, discharge, ... Charge and discharge are stored as **separate documents**, so unlike Severson we skip the "find where discharge starts inside the cycle" step entirely.

4.2 Per-cycle preprocessing function

```
from scipy.interpolate import interp1d
from scipy.integrate import cumulative_trapezoid

def inp_500(x, t):
    """Linear interpolation to 500 uniform timesteps, copied from
acctouhou."""
    f = interp1d(t, x, kind='linear')
    t_new = np.linspace(t.min(), t.max(), num=500)
    return f(t_new)

def preprocess_cycle(data: dict) -> np.ndarray:
    """Turn one NASA cycle into the (4, 500) tensor acctouhou expects.

    Channels: [Q, V, I, T]
    """
    t = np.asarray(data['Time'], dtype=float)          # seconds
    V = np.asarray(data['Voltage_measured'], dtype=float)
    I = np.asarray(data['Current_measured'], dtype=float)
    T = np.asarray(data['Temperature_measured'], dtype=float)

    # Derive cumulative capacity by Coulomb counting.
    # NASA current is in A; time is in seconds; we want Ah.
    # For discharge: current is negative, so take |I|.
    # Q(t) = ∫ |I| dt / 3600  (seconds → hours)
    Q = cumulative_trapezoid(np.abs(I), t, initial=0) / 3600.0

    # Guard: drop if too short to interpolate meaningfully
    if len(t) < 100:
        return None

    Q500 = inp_500(Q, t)
    V500 = inp_500(V, t)
    I500 = inp_500(I, t)
    T500 = inp_500(T, t)
    return np.stack([Q500, V500, I500, T500], axis=0)   # (4, 500)
```

4.3 Summary feature vector (UNKNOWN until we inspect `summary_norm.npy`)

The acctouhou code does:

```
def concat_data(x1, x2, x3):
    normalized_data = (x3 - summary_norm[0]) / summary_norm[1]
    return np.hstack((x1, x2, normalized_data))
```

`summary_data[cell][cycle]` is indexed per cycle and has dimension `S` such that `F_ch + F_dis + S = 12`. Likely candidates based on Severson paper conventions + the `chargetime` reference in `data_processing.ipynb`:

- `chargetime` — total charge duration this cycle (seconds)
- discharge `Capacity` — the scalar already in NASA data
- cycle-level average / min / max temperature

Action item (blocker for validation): download `summary_norm.npy`, inspect `shape` and `dtype`, reverse-engineer the field schema. Everything else in the pipeline is determined — this one `.npy` file tells us the last unknown.

If `S = 4`, the summary vector is almost certainly `[chargetime, Capacity, min_T, max_T]` or similar. We derive all of these from NASA raw data trivially.

4.4 Per-cell assembly

```
def preprocess_cell(cell_id: str, raw_path: Path):
    raw = json.load(open(raw_path))
    cycles = raw[cell_id]['cycle']

    charges    = [c for c in cycles if c['type'] == 'charge']
    discharges = [c for c in cycles if c['type'] == 'discharge']

    # Pair them: charge[i] with discharge[i]. NASA stores them alternating.
    # If counts differ, truncate to min.
    n = min(len(charges), len(discharges), 100)

    ch_tensors  = []
    dis_tensors = []
    summaries   = []
    for i in range(n):
        ch_t  = preprocess_cycle(charges[i]['data'])
        dis_t = preprocess_cycle(discharges[i]['data'])
        if ch_t is None or dis_t is None:
            # Skip cycles with too few samples; need contiguous 100 good
cycles
            continue
        ch_tensors.append(ch_t)
        dis_tensors.append(dis_t)
        summaries.append(build_summary(charges[i], discharges[i]))

    if len(ch_tensors) < 100:
        raise ValueError(f"{cell_id} has only {len(ch_tensors)} clean paired
cycles")
```

```
    return (
        np.stack(ch_tensors[:100]),   # (100, 4, 500)
        np.stack(dis_tensors[:100]),  # (100, 4, 500)
        np.stack(summaries[:100]),    # (100, S)
    )
```

4.5 Unit correctness — the silent failure mode

Normalization stats (`charge_norm.npy` etc.) were fitted on Severson data. If our NASA-derived arrays are in different units than Severson's, the normalized values land outside the trained distribution and predictions will be garbage even with clean code.

Known unit checks:

Quantity	Severson (from the `.mat` reader)	NASA (from README)	Action
Time	minutes	**seconds**	Convert NASA time: `t_sec / 60` before Q-integration and interpolation
Current	A	A	Match ✓
Voltage	V	V	Match ✓
Temperature	°C	°C	Match ✓
Q (derived)	Ah	Ah (after dividing by 3600)	Need `/3600` if time is in seconds

Expert: please confirm the Severson time unit. The `data_processing.ipynb` reads `t = f[cycles['t']...]`. Their `.mat` file convention is unclear without inspecting one. If Severson stores time in minutes and we feed seconds, interpolation still works (different t-axis), but the interp1d support range changes and Q values will be scaled 60× differently from training.

---

5. Architecture

```
src/servers/battery/
    __init__.py
    main.py                     # FastMCP tool wrappers (thin)
    couchdb_client.py           # fetch cycle docs from dedicated battery DB
    preprocessing.py            # the functions in §4: inp_500,
preprocess_cycle, preprocess_cell
    model_wrapper.py            # lazy TF load + inference; copies
feature_selector / concat_data / process2predict from acctouhou
    chemistries.yaml            # thresholds, nominal values
    model_weights/              # (gitignored) pretrained .h5 + .npy files
    sample_data/                # small subset for Docker bootstrap
    tests/
    scripts/
        fetch_weights.sh        # downloads from Google Drive
```

Data layer: dedicated `battery` CouchDB database (separate from `chiller`, `vibration`). `BATTERY_DBNAME` env var defaults to `battery`.

Document schema: one doc per cycle, preserving full within-cycle arrays:

```
{
  "asset_id": "B0018",
  "cycle_index": 47,
  "cycle_type": "discharge",
  "timestamp": "2008-05-30T10:23:42",
  "ambient_temperature": 24,
  "data": {
    "Voltage_measured": [4.18, 4.17, ...],
    "Current_measured": [-2.01, -2.01, ...],
    "Temperature_measured": [24.1, 24.2, ...],
    "Time": [0.0, 18.0, ...],
    "Capacity": 1.843
  }
}
```

No pre-staged CSV. No synthetic timestamps. No hardcoded filename in LLM prompts.

5.1 Environment variables

The plan reuses existing CouchDB env vars and adds a small number of battery-specific ones. Verified against `.env.public` and existing server conventions (`vibration/couchdb_client.py`, `couchdb_setup.sh`, `docker-compose.yaml`).

Reused (no change to `.env.public`):

Variable	Purpose	Notes
`COUCHDB_URL`	CouchDB endpoint	Shared across all DB servers
`COUCHDB_USERNAME` / `COUCHDB_PASSWORD`	CouchDB credentials	Shared
`LOG_LEVEL`	Server log verbosity	Already honored by every server's `main.py`
`WATSONX_APIKEY` / `WATSONX_PROJECT_ID` / `WATSONX_URL`	LLM for `diagnose_battery`	Same pattern as FMSR
`LITELLM_API_KEY` / `LITELLM_BASE_URL`	Alternative LLM provider	If not using WatsonX
`FMSR_MODEL_ID`	LLM model identifier	`diagnose_battery` can share this var or have its own

New variables to add to `.env.public`:

Variable	Default	Purpose
`BATTERY_DBNAME`	`battery`	Dedicated CouchDB database for battery documents. Already referenced in `docker-compose.yaml:10` and `couchdb_setup.sh:59` but missing from `.env.public` — needs to be added for dev workflows outside Docker
`BATTERY_MODEL_WEIGHTS_DIR`	`src/servers/battery/model_weights`	Directory holding the downloaded pretrained `.h5` + `.npy` files. The server lazy-loads from this path on first model tool call
`BATTERY_DATA_DIR`	`.data/5. Battery Data Set_json`	Path to the raw NASA JSON files, used by the CouchDB init script to seed the `battery` DB. Only needed at ingest time, not at server runtime
`BATTERY_MODEL_ID`	inherits from `FMSR_MODEL_ID`	Optional override for the LLM used in `diagnose_battery`. Kept separate so we can tune narration style without affecting FMSR

Dependency group in `pyproject.toml`:

```
[dependency-groups]
battery = [
    "tensorflow>=2.15",   # loads acctouhou .h5 weights
    "scikit-learn>=1.3",  # already used elsewhere
    # scipy, numpy, pandas already in core deps
]
```

Installed via `uv sync --group battery`. Server starts without this group installed — all TF imports are lazy inside tool functions, so `list_batteries`, `get_battery_cycle_summary`,

`analyze_impedance_growth`, and `detect_capacity_outliers` work without TensorFlow. Only `predict_*` tools and `diagnose_battery` require the group.

Proposed `.env.public` additions:

```
# ── Battery server ──────────────────────────────────────────────
BATTERY_DBNAME=battery
BATTERY_MODEL_WEIGHTS_DIR=src/servers/battery/model_weights
BATTERY_DATA_DIR=.data/5. Battery Data Set_json
# BATTERY_MODEL_ID=                  # optional; inherits FMSR_MODEL_ID if unset
```

Runtime checks the server should perform at startup (fail loudly, not silently):

1. If `BATTERY_MODEL_WEIGHTS_DIR` is set but the directory is missing the four expected `.h5` files and four `.npy` files → log a clear warning and mark pretrained-model tools as unavailable. Statistical tools still work.
2. If `COUCHDB_URL` is unreachable → the server still starts (MCP protocol doesn't require backends), but tool calls return `ErrorResult` with guidance.
3. If `BATTERY_DBNAME` is set but the database doesn't exist → return a setup-hint error pointing to `couchdb_setup.sh`.

---

6. Tool surface (8 tools)

Tool	Purpose	Uses pretrained model?	Usable cells
`list_batteries(site_name)`	Discover asset IDs	No	34
`get_battery_cycle_summary(asset_id)`	Per-cycle capacity/voltage/Rct/temperature table	No	34
`predict_rul(asset_id)`	RUL cycles + survival, uses `predictor.h5`	Yes	10
`predict_voltage_curve(asset_id, cycle_index)`	V-SOC curve for a cycle, uses `predictor2.h5`	Yes	10
`predict_voltage_milestones(asset_id, thresholds=[2.9,2.8,2.7])`	Cycle where voltage first crosses each threshold	Yes	10
`analyze_impedance_growth(asset_id)`	Exponential fit to Re/Rct + alarm state	No	34
`detect_capacity_outliers(asset_ids, window=50)`	Fleet z-score outliers	No	34

Tool	Purpose	Uses pretrained model?	Usable cells
`diagnose_battery(asset_id)`	One-shot: RUL + impedance + outlier → LLM-narrated report	Yes + LLM	10

Tools that **stay in TSFM** (genuinely generic):

- `analyze_sensitivity` — Pearson correlation + quantile binning (for thermal/ambient scenarios)
- Performance-metrics exposure in `run_tsfm_forecasting`

Tools to **delete** (duplicative once battery server exists):

- `estimate_remaining_life` in TSFM — `predict_rul` supersedes it

---

7. Domain knowledge — `chemistries.yaml`

Single source of truth. Loaded at server startup into module constants.

```
li_ion_nca_18650:          # NASA B0xx default
  nominal_capacity_ah: 2.0
  eol_threshold_soh: 0.80
  eol_capacity_ah: 1.4                  # Scenario 1: 30% fade threshold
  cutoff_voltage_min: 2.7
  voltage_milestones: [2.9, 2.8, 2.7]
  rct_alarm_multiplier: 1.5             # vs. initial Rct
  re_alarm_multiplier: 2.0
  ce_anomaly_drop_pct: 2.0              # Coulombic efficiency drop →
plating
  knee_detection_slope_ratio: 2.0       # 2x slope change flags knee
  temperature_alarm_c: 45
```

Expert, please review: are these thresholds the right defaults for NASA NCA 18650 assessment? What's missing?

---

8. LLM-as-tool for `diagnose_battery`

Uses the FMSR pattern — one direct LiteLLM call from inside the tool. No nested planner.

Pipeline

1. Fetch cycles from CouchDB.
2. Run numerical analysis:
   - `predict_rul` → RUL cycles
   - `analyze_impedance_growth` → Rct growth rate, alarm
   - Statistical: capacity fade rate, CE drop detection

3. Build prompt: numerical results + thresholds from `chemistries.yaml` + **exactly 3 few-shot examples**.
4. Call LiteLLM, parse JSON.
5. Return combined result (numbers + narration + recommendations).

The 3 few-shot examples

```
- input: {SOH: 0.84, rct_growth_per_50cy: 0.03, ce_stable_at: 0.992,
          knee_detected: false, knee_cycle: null}
  output:
    primary_mode: "capacity_fade"
    severity: "routine"
    explanation: "Linear SEI-driven fade within expected envelope for NCA."

- input: {SOH: 0.89, rct_growth_per_50cy: 0.04, ce_min: 0.968,
          ce_drop_cycle: 140, fade_accelerating: true}
  output:
    primary_mode: "lithium_plating"
    severity: "alarm"
    explanation: "CE drop >2% with accelerating fade indicates plating,
                  typically from fast-charge at low temperature."

- input: {SOH: 0.91, rct_growth_per_50cy: 0.18, ce_stable_at: 0.990,
          fade_linear: true}
  output:
    primary_mode: "impedance_growth"
    severity: "alarm"
    explanation: "DCIR rising far faster than capacity fade suggests
electrode
                  degradation; elevated fire risk in second-life
applications."
```

Why exactly 3: disambiguates the three distinct failure signatures without diluting the prompt with surface-level pattern matching.

Expert, please review: do these 3 examples cover the right distinct signatures? Any third-rail failure mode we'd miss?

---

9. Scenario coverage (15 entries in `battery_scenarios.json`)

#	Scenario	Primary tool	Cells that work
1	Fleet EV operator — RUL to 30% fade	`predict_rul` over fleet	10
2	BMS — EOD prediction	`predict_voltage_milestones`	10

#	Scenario	Primary tool	Cells that work
3	Warranty variability	`predict_rul` fleet + std dev	10
4	Thermal/charging correlation	`analyze_sensitivity` (TSFM)	34
5	Edge AI lightweight	`predict_rul` with inference timing	10
6	EOD short-horizon	`predict_voltage_milestones`	10
7	Logistics RUL ranking	`predict_rul` fleet	10
8	Voltage milestones (2.9/2.8/2.7V)	`predict_voltage_milestones`	10
9	QA anomaly detection	`detect_capacity_outliers`	34
10	Ambient temperature impact	`analyze_sensitivity` + grouped `predict_rul`	34 / 10
11	Second-life resistance	`analyze_impedance_growth` + `predict_rul`	34 / 10
12	Early thermal-runaway warning	`analyze_impedance_growth` + `detect_capacity_outliers`	34
13	Partial-data RUL (first 25%)	`predict_rul` with window_end=N*0.25	**May fail** — model needs ≥100 cycles of history; 25% of a 132-cycle cell is 33 cycles
14	Sensor drift	`analyze_impedance_growth` residuals	34
15	Baseline vs advanced benchmarking	`predict_rul` + TSFM forecast side-by-side	10

Scenario 13 gotcha: the acctouhou predictor uses cycles 100+ as starting points for inference. "First 25% of a 132-cycle cell = 33 cycles" is below that floor. Options: (a) drop scenario 13 from v1, (b) use TSFM for just scenario 13, (c) demonstrate scenario 13 only on B0033/34/36/B0005–07 (197 or 168 cycles → 49 or 42 cycles = still below 100).

Scenario 13 mitigation: document it as a "use TSFM, not the pretrained model" case. The goal of the scenario is to show RUL from partial data — TSFM forecasts can answer this even with short histories, even if at lower accuracy. Honest limitation, not a blocker.

---

10. What we explicitly will NOT do

- **No PBT integration in v1.** PBT requires fine-tuning per dataset, violates the "pretrained only" requirement.
- **No in-memory data cache.** Per-cell cycle counts are ≤200; CouchDB fetch per call is cheap. Add only if benchmarking shows it matters.
- **No MCP resources/prompts primitives.** Our plan-execute runner doesn't consume them.
- **No nested planner inside `diagnose_battery`.** Direct LiteLLM call (FMSR pattern).
- **No synthetic timestamps, no pre-staged CSV.** Delete `src/couchdb/prepare_battery_csv.py` and `artifacts/output/tuned_datasets/battery_cycle_metrics.csv`.
- **No chemistry beyond Li-ion NCA 18650 in v1.** YAML is extensible, but we target NASA B0xx only.
- **No re-training the acctouhou weights.** Use as-is, accept the LFP-training-domain constraint, validate accordingly.
- **No baking the 15 scenarios into prompts as few-shot.** They are the eval set.

---

11. Validation plan

This is a **go/no-go gate** before productionizing the pretrained-model tools. Do it in a one-off notebook, not in the server.

11.1 Reproduce published Severson results

Before touching NASA, confirm our TF version loads acctouhou's weights and reproduces their reported MAE:

1. Install `tensorflow>=2.15`, `scikit-learn`, `numpy`, `scipy`.
2. Run their `predict.py` on their bundled Severson `.npy` files.
3. Expected: `testing_RUL_mae` within ~95 cycles (per paper Table 2).

If their pipeline doesn't load/run in our TF version, we need to pin an older TF in the `battery` extras group or containerize.

11.2 Inspect `summary_norm.npy`

Print `shape`, `dtype`, first rows. Document the summary feature schema. Update §4.3 with the confirmed schema.

11.3 NASA preprocessing + inference on B0018

B0018 is the cleanest single-cell candidate (132 discharge cycles, standard 2A profile, 24°C).

1. Run preprocessing from §4 on B0018 → produce `(1, 100, 4, 500)` charge + discharge + summary tensors.
2. Apply Severson normalization.
3. Run through feature_selector_ch + feature_selector_dis + concat.
4. Run predictor; denormalize.
5. Compare predicted RUL to ground truth (B0018 has 132 discharge cycles; EOL at 1.4 Ah is around cycle 60–70 per the fade curve).

Pass criterion: MAE on held-out cycles ≤ 30 cycles (roughly ±30% of typical RUL).

If it fails:

- Option A — model doesn't transfer from LFP to NCA. Accept this, fall back to TSFM + statistical curve fits for `predict_rul`. Keep the pretrained tools in a "research" mode off by default.
- Option B — preprocessing has a units/scale bug. Iterate.
- Option C — summary features are wrong. Iterate after §11.2 inspection.

11.4 Expert review

Show outputs to the battery expert for sanity check before any MCP wiring.

---

12. Implementation sequence

1. **Validation (steps 11.1 → 11.3).** No code in the server yet. One-off notebook.
2. **Download pretrained weights** via `scripts/fetch_weights.sh`. Gitignore `model_weights/`.
3. **Ingest raw NASA JSON into dedicated `battery` CouchDB** via `couchdb_setup.sh`. Filter out discharge documents with `<100` samples at load time; log counts.
4. **Scaffold `src/servers/battery/`** — register in `pyproject.toml` and `DEFAULT_SERVER_PATHS`.
5. **Implement non-model tools first** — easier to debug: `list_batteries`, `get_battery_cycle_summary`, `analyze_impedance_growth`, `detect_capacity_outliers`.
6. **Implement preprocessing + model tools** — `preprocessing.py`, `model_wrapper.py`, `predict_rul`, `predict_voltage_curve`, `predict_voltage_milestones`.
7. **Implement `diagnose_battery`** with LLM narration + 3 few-shot examples.
8. **Delete TSFM's `estimate_remaining_life`**, delete `prepare_battery_csv.py`, delete `battery_cycle_metrics.csv`.
9. **Run all 15 scenarios end-to-end** through `plan-execute`. Verify planner routes to `battery` server correctly.
10. **Expert review** of tool outputs. Tune thresholds in `chemistries.yaml`.

---

13. Open questions for the battery expert

1. **LFP→NCA transferability.** The acctouhou model was trained on Severson LFP cells (2.0–3.6V range, ~1.1 Ah nominal). NASA is NCA (2.7–4.2V, ~2.0 Ah). How much accuracy degradation do you expect from the chemistry mismatch? Is ±30% RUL MAE a reasonable acceptance threshold for v1?

2. **Severson time units.** Does the Severson `.mat` store `t` in minutes or seconds? This determines whether the `charge_norm.npy` scale was fitted on minute-scale or second-scale arrays, and whether we need to convert NASA seconds to minutes before interpolation.

3. **Summary feature schema.** After we inspect `summary_norm.npy`, are fields like `[chargetime, Capacity, min_T, max_T]` the expected derivation targets? Any we'd miss?

4. **Coulomb-counting direction.** For Q(t) we use `|I|` integrated over time. Severson uses `Qd` and `Qc` which are cumulative positive quantities. Is taking `abs()` the right convention, or should we keep signed integration?

5. **Thresholds in `chemistries.yaml`.** Are 1.5× Rct, 2.0× Re, 2% CE drop the right starting alarm points for NCA 18650?

6. **The 3 few-shot examples.** Do they cover the right distinct signatures? Any failure mode we'd miss at 3 examples?
7. **Scenario 13 (partial-data RUL).** The pretrained model structurally can't do this (needs ≥100-cycle history). Is the proposed fallback — TSFM forecast for this scenario only — acceptable, or do we need a dedicated early-cycle RUL method?
8. **Scenario 12 (thermal runaway precursor).** Plan detects this via impedance growth + capacity outliers. Sufficient, or do we need a dedicated thermal model?
9. **Knee-point detection method.** Plan uses 2.0× slope ratio. Would you prefer bacon-watts segmented regression, second-derivative zero-crossing, or something else?
10. **Acctouhou licensing.** Repo has Apache 2.0 (`LICENSE.txt`). Confirms: compatible for our use.

---

14. Per-cell TSFM compatibility (for scenarios that fall back to TSFM)

Cell	Discharge cycles	`ttm_52_16` infer	`ttm_90_30` infer	`ttm_96_28` infer	`ttm_96_28` finetune (≥124)
B0005, B0006, B0007	168	✅	✅	✅	✅
B0018	132	✅	✅	✅	✅
B0033, B0034, B0036	197	✅	✅	✅	✅
B0054, B0055, B0056	102	✅	✅	✅	❌
B0045–B0048	72	✅	❌	❌	❌
B0041 (clean)	25	❌	❌	❌	❌
B0029–B0032	40	❌	❌	❌	❌

Implication for scenario 13 (partial-data RUL from first 25% of cycles): if we use `ttm_52_16_mae`, we can feed the first ~55 cycles of B0005–07/B0018/B0033–36 and get a 16-cycle forecast. The acctouhou pretrained model cannot handle this since it needs ≥100 cycles before producing any output, so `ttm_52_16_mae` is the right tool for that specific scenario.

---

15. Summary for the expert (TL;DR)

- 34 NASA cells available, 10 have enough data for the pretrained model, 24 usable for statistical tools.
- Pretrained model expects `(n_cells, 100, 4, 500)` = `[Q, V, I, T]` per cycle at 500 interpolated timesteps, then a 50-cycle sliding window → predictor.
- Preprocessing from NASA raw is mechanically straightforward: Coulomb-count to get Q, linear interp to 500, apply their normalization, run their model. The **chemistry mismatch (LFP → NCA)** is the only real unknown.

- Before building any MCP tool, validate on B0018. If RUL MAE ≤ 30 cycles, ship. Otherwise fall back to TSFM for RUL tools.
- 8 tools total in the new server. 3 use the pretrained model; 4 are statistical (no ML); 1 is an orchestrating diagnostic that chains them with an LLM narration.
- We're not modifying the generic TSFM server further — its `analyze_sensitivity` and performance-metrics exposure stay; `estimate_remaining_life` gets deleted.